

COMPLETE PROJECT EXPLANATION SCRIPT

Automated Incident Root-Cause Analysis via Close-Code Classification

INTRODUCTION

Good morning everyone.

Today I am presenting my project titled:

"Automated Incident Root-Cause Analysis via Close-Code Classification."

This project focuses on using **Machine Learning and Natural Language Processing** to assist in identifying probable root causes of IT incidents automatically.

WHAT IS INCIDENT MANAGEMENT?

Before understanding the project, first let us understand what an incident is.

In companies, especially IT companies, many issues occur daily such as:

- Email not working
- VPN connection failed
- Server downtime
- Database timeout
- Network outage
- Login authentication failure

Whenever such a problem occurs, an **incident ticket** is created in an Incident Management System like:

- ServiceNow
- Jira Service Management
- BMC Remedy

These tickets contain information such as:

- Incident description
 - Priority
 - Category
 - Assignment group
 - Time of occurrence
 - Resolution details
-

WHAT IS A CLOSE CODE?

After the issue is solved, support engineers assign a final reason called a **Close Code**.

For example:

Incident	Possible Close Code
VPN not connecting	Network Failure
Login failed	Authentication Issue
Application crashed	Software Failure
Database timeout	Backend Failure

The close code represents the final root cause of the incident.

PROBLEM IN EXISTING SYSTEM

Currently, close-code assignment is mostly manual.

This creates several problems:

- Time-consuming
- Depends on engineer experience
- Inconsistent classification
- Difficult for large enterprises handling thousands of incidents

For example:

Suppose an employee reports:

“Email service is not working.”

This could actually happen because of:

- Network issue
- SMTP server issue
- Authentication failure
- Backend server issue

The engineer manually investigates everything.

This process consumes a lot of time.

AIM OF THE PROJECT

The goal of this project is:

To automatically predict probable close codes using Machine Learning.

Instead of forcing one exact answer, the system gives:

- Top recommendations
- Confidence scores
- Most probable root causes

This acts like an intelligent assistant for support engineers.

DATASET USED

For this project, I used the:

“Incident Management Process Enriched Event Log Dataset”

from the UCI Machine Learning Repository.

This dataset contains real-world enterprise incident records.

Initially, the dataset had more than:

- 140,000 records

and many incident lifecycle attributes.

WHAT DATA WAS AVAILABLE?

The dataset included fields such as:

- Incident state
 - Priority
 - Urgency
 - Impact
 - Assignment group
 - Reassignment count
 - Reopen count
 - Close code
 - Incident descriptions
-

MAJOR CHALLENGE IN DATASET

One important challenge was:

Event Log Structure

One incident appeared multiple times because incidents move through different stages:

- New
- Active
- Resolved
- Closed

So first I had to aggregate multiple records into one final incident record.

DATA PREPROCESSING

Data preprocessing was one of the most important parts of this project.

The preprocessing steps included:

1. Handling Missing Values

Some fields were empty or unknown.

Example:

Priority	Category
NULL	Network

These missing values were cleaned or replaced.

2. Removing Duplicate Records

Duplicate incidents were removed because duplicates can bias the model.

3. Rare Class Filtering

Some close codes had only 2 or 3 records.

Machine Learning models cannot learn properly from extremely small classes.

So rare classes were removed.

4. Dataset Balancing

The dataset was highly imbalanced.

Example:

Close Code	Count
Code 6	14,000+
Code 12	2

This causes models to predict only majority classes.

So balancing techniques were applied.

FEATURE ENGINEERING

Feature engineering means creating useful features for the model.

I used two types of features:

STRUCTURED FEATURES

These are normal columns like:

- Priority
 - Impact
 - Urgency
 - Reassignment count
 - Reopen count
-

EXAMPLE

Suppose:

Incident	Reassignment Count
VPN failure	1
Complex database outage	8

Higher reassignment counts may indicate more complex incidents.

So this becomes useful information for the model.

TEMPORAL FEATURES

Time-based features were also created such as:

- Opened hour

- Opened day
- Incident duration

Example:

If many incidents happen during office hours, patterns can emerge.

NATURAL LANGUAGE PROCESSING (NLP)

One of the most important parts of the project was NLP.

Incident descriptions are text data.

Machine Learning cannot directly understand text.

So I used:

TF-IDF Vectorization

SIMPLE EXPLANATION OF TF-IDF

Suppose these incidents exist:

1. "VPN connection failed"
2. "VPN timeout issue"
3. "Database server failed"

The word "VPN" becomes important for VPN-related incidents.

TF-IDF converts these words into numerical vectors.

So the model can mathematically process incident descriptions.

MACHINE LEARNING MODELS USED

I implemented four models:

1. Logistic Regression
 2. Decision Tree
 3. Support Vector Machine (SVM)
 4. Random Forest
-

WHY MULTIPLE MODELS?

Different models behave differently.

Some models work better for:

- Text data
- Sparse data
- Multiclass classification

So comparison is important.

LOGISTIC REGRESSION

This was used as a baseline model.

Advantages:

- Fast
 - Simple
 - Good for text classification
-

DECISION TREE

Decision Trees work like flowcharts.

Example:

IF:

- Priority = High
- Category = Network

THEN:

- Possible root cause = Network Failure

They are easy to understand but may overfit.

SVM (SUPPORT VECTOR MACHINE)

SVM performed best in this project.

It works well for:

- High-dimensional data
- TF-IDF vectors
- Sparse datasets

The highest observed performance came from SVM.

RANDOM FOREST

Random Forest combines many Decision Trees.

Advantages:

- Better generalization
 - Reduced overfitting
 - More stable predictions
-

MODEL EVALUATION

The models were evaluated using:

- Accuracy
 - Precision
 - Recall
 - F1-score
-

BEST RESULT

The best observed performance was approximately:

- Accuracy $\approx$ 59%
- F1-score $\approx$ 52%

using Random Forest

IMPORTANT QUESTION:

WHY IS ACCURACY NOT 90%?

This is a very important discussion point.

The dataset is a real-world enterprise dataset.

Real incidents are highly ambiguous.

EXAMPLE OF AMBIGUITY

Suppose the incident says:

“Email service not working”

This could actually mean:

- Network issue
- SMTP issue
- Authentication failure
- Backend server issue

Even humans may disagree on the exact close code.

So achieving extremely high accuracy is difficult in real operational datasets.

VERY IMPORTANT POINT

This system should NOT be treated as:

A perfect deterministic prediction engine.

Instead, it should be treated as:

An intelligent recommendation system.

TOP-K RECOMMENDATION SYSTEM

Instead of giving only one prediction, the system gives:

- Top-1 recommendation
- Top-3 recommendations
- Confidence scores

Example:

Rank	Root Cause	Confidence
1	Network Failure	42%
2	Authentication Issue	31%
3	SMTP Failure	18%

This is more practical for enterprise environments.

STREAMLIT DEPLOYMENT

After training the model, I developed a Streamlit interface.

The interface allows users to:

- Select category
- Select priority

- Enter symptom description
 - Get root-cause recommendations
-

EXAMPLE INPUT

Suppose the user enters:

- Category = Software
- Priority = High
- Description = "Database connection timeout"

The system predicts:

1. Database Failure
2. Backend Service Failure
3. Server Connectivity Issue

with confidence scores.

CHALLENGES FACED

Major challenges included:

- Imbalanced dataset
 - Sparse textual data
 - Ambiguous incident descriptions
 - Overlapping categories
 - Moderate model generalization
-

WHAT I LEARNED

This project helped me understand:

- Machine Learning pipelines
- NLP implementation
- Feature engineering
- Model evaluation

- Recommendation systems
 - Streamlit deployment
 - Real-world ITSM analytics
-

FUTURE ENHANCEMENTS

Future improvements may include:

- BERT embeddings
 - Semantic sentence transformers
 - Explainable AI
 - Real-time ServiceNow integration
 - Cloud deployment
 - Deep learning models
-

FINAL CONCLUSION

To conclude:

This project successfully demonstrated how Machine Learning and NLP can assist in enterprise incident analysis.

The system acts as an:

AI-assisted intelligent recommendation system

for predicting probable incident close codes and helping support engineers perform faster root-cause analysis.

Thank you.